Usage walkthrough

One scenario, start to finish: build the environment, generate the test data, open a capture, apply an interpretation, and produce a report. Every step lists the command and what it should produce. Paths are relative to the repository root.

The tool has three stages, each usable on its own:

- `src.capture` - PCAP/PCAPNG in, normalized capture (sessions, directional streams, byte provenance) out.
- `src.protocol` - a declarative rule applied to a normalized capture, giving per-message results and statuses.
- `src.project` - a portable project directory, the end-to-end pipeline, and the report.

1. Environment

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e ".[dev]"
```

Expected: the package installs and `pytest` becomes available. Python 3.12 or newer, Linux x86-64.

2. Test data

The corpus is committed under `tests/corpus/`. Regenerate it only if you change the generators:

```
python -m scripts.generate_corpus
```

Expected: `tests/corpus/corpus_capture_01.pcapng`, `corpus_capture_02.pcapng`, `corpus_capture_defects.pcapng`, `synthetic_live.pcapng`, and their journals are written. The reference exports under `tests/corpus/reference_export/` are unchanged by a clean run; `git status` should stay clean.

3. Normalize a capture

```
python -m src.capture.cli \
  --pcap tests/corpus/corpus_capture_01.pcapng \
  --out out/cap01.json \
  --source-name captures/corpus_capture_01.pcapng
```

Expected: a one-line JSON summary on stdout with `capture_id`, `sessions`, and `diagnostics`; `out/cap01.json` holds the normalized capture with, for each session and direction, `bytes_b64`, `provenance`, and `diagnostics`.

Add `--stream` to process a large capture in blocks, or `--verify-checksums` to report TCP checksum mismatches instead of ignoring them. Add `--wireshark-pcap out/reassembled.pcap` to write the rebuilt streams as a pcap for inspection in Wireshark.

4. The end-to-end pipeline

One command takes a capture to a project, a normalized capture, an optional result, and a report:

```
python -m src.project.cli pipeline \
  --pcap tests/corpus/corpus_capture_01.pcapng \
  --project project.madrigal \
  --report reports/REPORT.md
```

Expected: a JSON summary listing the project, the normalized capture, the session count, and the report paths. The project directory looks like:

```
project.madrigal/
  manifest.json
  captures/corpus_capture_01.pcapng
  results/normalized.json
  reports/REPORT.md
  reports/REPORT.html
  logs/  rules/  annotations.sqlite
```

Without a rule, the report says plainly that no rule was applied. Add a rule to describe the message structure:

```
python -m src.project.cli pipeline \
  --pcap tests/corpus/corpus_capture_01.pcapng \
  --project project.madrigal \
  --rule src/protocol/examples/set_parameter_request.json \
  --force
```

Expected: `results/result.json` appears with a per-message `status` and a `summary` of status counts, and the report gains a hypothesis, a counterexample list, and open questions. The counterexamples are the messages the rule could not confirm; each is tied to an offset in the stream.

5. Project management

```
python -m src.project.cli create --path /tmp/lab/project.madrigal --name "parameter study"
python -m src.project.cli add-capture --project /tmp/lab/project.madrigal --pcap tests/corpus/
corpus_capture_02.pcapng
python -m src.project.cli open --path /tmp/lab/project.madrigal --show
python -m src.project.cli export --project /tmp/lab/project.madrigal --out /tmp/lab.zip
python -m src.project.cli import --in /tmp/lab.zip --target /tmp/lab-restored
```

Expected: `create` makes the directory, `add-capture` copies the file and records its sha256, `open --show` prints the manifest, `export` writes a zip, and `import` unpacks it and reports `"broken": []` when every capture still matches its digest.

The portability check is the point of `export`/`import`: after import the research opens and verifies without any absolute path from the original machine.

6. Applying a rule on its own

When you already have a normalized capture, apply a rule without a project:

```
python -m src.protocol.cli apply \
  --rule src/protocol/examples/set_parameter_request.json \
  --capture out/cap01.json \
  --out out/result.json
```

Expected: `out/result.json` follows the result contract: `rule_id`, `rule_version`, `capture_id`, `messages`, and `summary`. This is the same result the pipeline writes, and the shape a downstream tool reads.

7. The interface

The PySide6 interface reads the same artifacts: a normalized capture from `src.capture`, and a result from `src.protocol`. Select a session and direction, inspect the bytes with their packet provenance, and open a rule and its results. The report and the machine-readable result are the durable output; the interface is a viewer over them.

A capture can be opened two ways from `File`: `Open normalized capture` reads a normalized JSON or a raw `.pcap`/`.pcapng`, and `Import PCAP` additionally writes the normalized document next to the input as `<name>.normalized.json`. Before anything is open the window shows an empty state with the order to follow — open a capture, pick a session, describe the messages, apply the rule, export the report — and marks each step as it is done; `Help -> Show start guide` brings the empty state back over the workspace.

A rule can be written by hand in the `Rule` tab of the interpretation panel, or built from a form with `File -> Build rule...` (`Ctrl+B`). The builder covers the framing (length-prefixed, fixed-size, marker-based, manual) and one row per field (name, offset, type, byte order, length, enum values, hypothesis flag, expected values), and shows the generated document live; when the builder is opened with a byte selected, the bytes of the message under the cursor are shown beside the form. The result is loaded into the editor, where attributes the form does not cover can still be added.

The interpretation panel reports where the rule stands: the loaded rule and its framing, whether it has been applied to the current direction, the last verdict counts, and — for the byte under the cursor — the field that covers it and the chain back to the source packet. `Rule -> Check on corpus...` runs the rule over every capture in the project and lists the counterexamples in a separate window, each one a jump back to the byte in the hex view.

7.1 Interface language

The window ships with an English and a Russian interface. Choose the language under the `Language` menu: `Russian`, `English` or `System`. `System` follows the desktop locale and falls back to English when no catalogue matches. The choice is stored with `QSettings` and restored on the next launch; switching languages relabels the open window immediately without reloading the capture.

The translations live in `src/ui/locale/`. English is the source language, so its catalogue mirrors the source strings one for one and is filled by a script rather than translated by hand; Russian is compiled from `madrigal_ru.ts` into `madrigal_ru.qm`. Both `.qm` files are committed so the window works without a Qt toolchain. After adding or changing a string, refresh both catalogues:

```
scripts/update_translations.sh
```

The script scans the window sources, updates each `.ts` in place (keeping the hand-written Russian translations), fills the English catalogue from the source strings and recompiles both `.qm` files.

7.2 Settings

`View -> Settings...` (or `Ctrl+,`) opens the preferences dialog. It has four sections: `General` for the theme and interface language, `Editor` for the hex and tree font sizes and the offset column, `Paths` for the directories the file dialogs start in, and `Advanced` for the logging level and the hex diagnostics shading. `OK` applies and closes, `Apply` applies without closing, and `Cancel` restores whatever was active when the dialog opened.

Theme and language apply at once through the same managers the menus use. The values are stored with `QSettings` under `madrigal/protocol-lab`. The full list of keys is in `docs/SETTINGS.md`.

8. Tests

```
python -m pytest tests/ -q
```

Expected: all tests pass. The suite covers the parser, sessions, reassembly on the defect fixtures, the streaming and regular pipelines agreeing, the project container round trip, the report renderers, and the corpus integration over the committed records.

9. Install into the application menu

The window ships with an icon and a freedesktop desktop entry. Install them for the current user, without root:

```
pip install -e .
bash scripts/install_desktop.sh
```

The script copies `packaging/madrigal-protocol-lab.desktop` into `~/.local/share/applications` and the rendered PNGs into `~/.local/share/icons/hicolor/<size>x<size>/apps`, then refreshes the desktop and icon caches. The launcher runs `madrigal-lab`, the `gui-scripts` entry point declared in `pyproject.toml`, so install the package first. A fresh login session may be needed before the entry appears.

Expected: an entry named "Madrigal Protocol Laboratory" in the application menu, opening the window with the hexagon icon in the title bar. The icon master is `assets/icons/app/madrigal-protocol-lab.svg`; regenerate the PNGs and their hicolor copies with `python scripts/render_icons.py`.

Notes

- All commands run offline.
- The capture, project, and report stages run on their own. The rule engine in `src.protocol` is optional, so the `--rule` variant in section 4 and the standalone command in section 6 require that module; without it the pipeline still runs and writes a report that states no rule was applied.
- The measured resource figures are in `docs/BENCHMARK.md`.
- The contracts are in `docs/CONTRACT.md`; the JSON Schemas are in `docs/schemas/`.